

Thiết kế và kiểm chứng PA-MPPI trên companion computer với ArduPilot và Gazebo

Nguyễn Thành Lâm

Báo cáo tiến độ kỹ thuật

13 tháng 09 năm 2026 – evidence-first revision

Tóm tắt

Báo cáo trình bày quá trình chuyển tầng tránh vật cản từ AP_Avoidance của ArduPilot sang companion computer. Hệ thống nhận point cloud LiDAR 3D, tối ưu quỹ đạo bằng Model Predictive Path Integral (MPPI), sau đó truyền setpoint cho ArduPilot ở chế độ GUIDED. Ba mức triển khai được đánh giá: vanilla MPPI với lệnh vận tốc, PA-MPPI v0 với bản đồ occupancy ba trạng thái và perception cost, và nhánh rigid-body experimental với trạng thái $[p, q, v, \omega]$ cùng lệnh collective thrust/body rate. Bản velocity-level có time-indexed global reference đã đạt terminal trong ba lần chạy Gazebo/SITL (right-angle, narrow-gate và slalom, mỗi map một seed). Nhánh rigid-body đã vượt qua kiểm tra hover, quaternion, packet MAVLink và closed-loop clear-air offline, nhưng chưa qua hover gate live trong Gazebo. Kết quả hỗ trợ tính khả thi của kiến trúc, không phải chứng nhận robustness hay an toàn phần cứng.

Từ khóa: MPPI, PA-MPPI, quadrotor, ArduPilot, MAVLink, LiDAR 3D, Gazebo, companion computer.

Trạng thái bằng chứng tại ngày 13/09/2026: VERIFIED OFFLINE: 40/40 unit tests và benchmark analytical 3 map $\times$ 3 seed. VERIFIED GAZEBO/SITL, ONE SEED: velocity-level right-angle, narrow-gate và slalom. IMPLEMENTED, NOT LIVE-VERIFIED: rigid body-rate/thrust và PA-MPPI unknown-space ablation. PENDING GAZEBO: rigid hover/body-rate và C-wall ablation. PENDING EDGE/HARDWARE: runtime/thermal, sensor model, HIL và flight.

Mục lục

1	Bối cảnh và mục tiêu	3
2	Kiến trúc hệ thống	3
2.1	Phân tầng điều khiển	3
2.2	Tại sao dùng kiến trúc này?	3
2.3	Các module phần mềm	4
3	Vanilla MPPI velocity-level	4
3.1	Mô hình dự đoán	4
3.2	Tối ưu MPPI	5
3.3	Cost bám global path (bản velocity-level)	5
3.4	Đối chiếu toàn bộ cost với paper	6
4	PA-MPPI v0	6
4.1	Occupancy map ba trạng thái	6
4.2	Perception-aware objective	7
5	Rigid-body PA-MPPI experimental	7
5.1	State, control và hệ trục	7
5.2	Dynamics rời rạc	7
5.3	Frame contract	8
6	Tích hợp MAVLink và các điều kiện an toàn	8
6.1	Velocity target	8
6.2	Goal tương tác từ RViz	8
6.3	Thrust và body-rate target	9
7	Thiết kế kiểm thử	9
7.1	Automated tests	9
7.2	Offline obstacle scenario	9
8	Kết quả	10
9	Thảo luận	11
9.1	Mức độ đáp ứng yêu cầu mentor	11
9.2	Khoảng cách tối bản triển khai trong bài báo	11
9.3	Rủi ro kỹ thuật còn lại	11
10	Lộ trình kiểm chứng tiếp theo	12
11	Kết luận	12
A	Lệnh tái lập	12
A.1	Automated tests	12
A.2	Rigid-body offline gate	12
A.3	Rigid-body live gate trong Gazebo	13

1 Bối cảnh và mục tiêu

Pipeline tuần trước đưa obstacle vào AP_Avoidance thông qua MAVLink. Cách này yêu cầu companion computer chuyển dữ liệu cảm biến sang representation và vòng đời mà ArduPilot quản lý. Tuy nhiên, protocol không bắt buộc tạo persistent obstacle ID: OBSTACLE_DISTANCE_3D cho phép UINT16_MAX khi ID không biết [1]; target handler cũng không đọc field này [2]. Mentor đề xuất tách hoàn toàn local planner khỏi autopilot: companion computer tự xử lý perception, sinh trajectory và gửi lệnh điều khiển; ArduPilot chỉ đảm nhiệm ổn định bay và tracking setpoint.

Mục tiêu kỹ thuật của giai đoạn này gồm:

1. xây dựng baseline MPPI không phụ thuộc AP_Avoidance;
2. tích hợp LiDAR 3D, state estimation và MAVLink thành một vòng kín;
3. bổ sung perception-aware objective theo PA-MPPI [3];
4. tiến dần từ velocity-level model tới rigid-body model;
5. định nghĩa tiêu chí kiểm thử có thể tái lập trước khi chạy trên phần cứng.

Phạm vi hiện tại là Gazebo và ArduPilot SITL. Báo cáo không xem kết quả offline hoặc SITL là bằng chứng đủ cho flight test trên quadrotor thật.

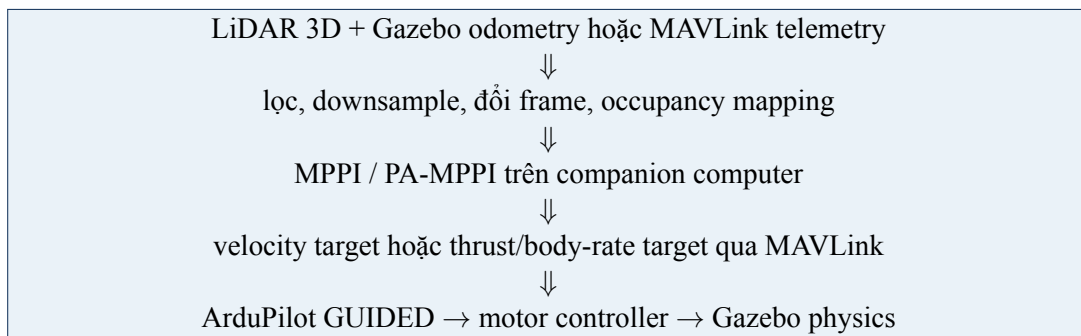
Định danh phiên bản và quy tắc nguồn

Project được audit tại commit 910c6a824fad0ca38615f93a0a17fc2c36d591cf. ArduPilot là dependency ngoài repository, không phải vendored/submodule, tại commit f808f78ce5a518ca96f2fb36420608b2b6254367; MAVLink gitlink tương ứng là 71d925850d7ddb01d0e56defc1046f9d9d3bf8ae. Protocol specification chỉ là thẩm quyền về message fields; source đúng commit là thẩm quyền cuối về hành vi firmware [4, 2]. Claim audit chi tiết nằm trong docs/SOURCE_AUDIT.md.

2 Kiến trúc hệ thống

2.1 Phân tầng điều khiển

Luồng dữ liệu triển khai trong repository được tóm tắt như sau:



Avoidance nội bộ của ArduPilot được tắt bằng OA_TYPE=0, AV0ID_ENABLE=0 và PRX1_TYPE=0. Do đó point cloud không đi qua OBSTACLE_DISTANCE_3D, AP_Proximity hoặc obstacle database của ArduPilot.

2.2 Tại sao dùng kiến trúc này?

ArduPilot cung cấp Guided setpoint interfaces và stabilization nội bộ [5]; đồng thời có proximity/Simple Avoidance và BendyRuler [6, 7]. **PROJECT DESIGN DECISION:** perception, occupancy mapping và PA-MPPI nằm ở companion vì optimizer cần truy cập trực tiếp map ba trạng thái và perception objective [3]. Việc tắt AP OA trong experiment tránh hai avoidance loop cùng sửa lệnh; nó không có nghĩa AP avoidance vốn không dùng được.

Bảng 1: Nguồn ràng buộc so với lựa chọn engineering.

Decision	Source-supported constraint	Our design choice / why
Companion planning	Guided nhận external setpoints; PA-MPPI dùng online map.	Giữ map/optimizer ngoài AP để không dịch internal representation.
Disable AP OA	AP OA là một stack độc lập.	Tắt trong ablation để tránh hai avoidance loop.
Velocity baseline	Guided hỗ trợ velocity/yaw-rate.	Reduced model để tách planner khỏi low-level validation.
Three-state map	Paper dùng unknown/free/occupied.	Fixed NumPy/Torch prototype; không gọi là ROG-Map.
Perception/unknown	Paper reward frontier và phạt non-free collision.	Port endpoint ray/LOS; giữ unknown collision policy.
Rigid branch	Paper dùng collective thrust/body rates.	Reduced closed-loop model, experimental only.
MAVLink interfaces	Spec/source hỗ trợ local velocity và target source nhận full rate vector.	Mask 1479 cho baseline; mask 128 + GUID=8 gate cho rigid.
50 Hz/rollouts	Paper: 50 Hz, 17,500 rollouts.	Rigid target 50 Hz, 512 rollouts do compute budget; phải benchmark live.
Safety gates	Low-level command giảm margin bảo vệ.	GUID readback, stale/brake/timeout, NaN và saturation gates.

2.3 Các module phần mềm

Bảng 2: Cấu trúc phần mềm chính.

Module	Trách nhiệm
mppi_controller.py	Vanilla MPPI, state 7 chiều và velocity command.
occupancy_grid.py	Voxel map unknown/free/occupied và ray integration.
pa_mppi_controller.py	Perception-aware cost trên velocity-level model.
rigid_body_pa_mppi.py	Rigid-body rollout, quaternion và thrust/body-rate command.
mavlink_interface.py	Telemetry, velocity target và SET_ATTITUDE_TARGET.
mppi_local_planner_node.py	Vòng điều khiển, waypoint, safety gate, diagnostics và RViz.

3 Vanilla MPPI velocity-level

PROJECT DESIGN DECISION: đây là tầng A trong staged validation. Nó kiểm tra local planning trong khi vẫn giao attitude và motor stabilization cho ArduPilot. Bằng chứng bên ngoài chỉ hỗ trợ việc Guided nhận velocity targets [5, 4]; mô hình giảm bậc này không được quy cho paper PA-MPPI.

3.1 Mô hình dự đoán

Baseline giữ ArduPilot ở tầng velocity tracking. State và control là

$$\mathbf{x} = [\mathbf{p}^\top, \mathbf{v}^\top, \psi]^\top \in \mathbb{R}^7, \quad \mathbf{u} = [\mathbf{v}_{cmd}^\top, \dot{\psi}_{cmd}]^\top \in \mathbb{R}^4. \quad (1)$$

Mô hình point-mass với first-order velocity lag được rời rạc hóa thành

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \alpha(\mathbf{v}_{cmd,k} - \mathbf{v}_k), \quad \alpha = \min(\Delta t/\tau, 1), \quad (2)$$

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_{k+1}, \quad (3)$$

$$\psi_{k+1} = \psi_k + \Delta t \dot{\psi}_{cmd,k}. \quad (4)$$

Đây là mô hình giảm bậc có chủ ý. Attitude, motor dynamics và lực khí động được đóng bên trong ArduPilot và Gazebo thay vì tối ưu trực tiếp. State rollout được mở rộng thêm setpoint khả thi trước đó $\mathbf{u}_{k-1}^{applied}$; low-pass, slew-rate limit và saturation giống tầng MAVLink được áp dụng bên trong dynamics trước khi tính $\mathbf{v}_{k+1}$. Đây là PROJECT DESIGN DECISION để prediction không giả định bước nhảy control mà interface thực không gửi được.

3.2 Tối ưu MPPI

Tại mỗi chu kỳ, N control sequence được tạo bằng cách thêm nhiều Gaussian vào nominal sequence. Với tổng cost S_j của rollout thứ j , trọng số là

$$w_j = \frac{\exp(-(S_j - S_{\min})/\lambda)}{\sum_{i=1}^N \exp(-(S_i - S_{\min})/\lambda)}. \quad (5)$$

Control sequence mới là trung bình có trọng số của các rollout. Cost baseline gồm khoảng cách tới waypoint, obstacle clearance, control effort, độ thay đổi control, yaw alignment và terminal distance. Obstacle cost dùng softplus quanh safe margin để giữ gradient chỉ phí mềm nhưng vẫn xử lý được point cloud rời rạc. Sampling, importance weighting và receding-horizon update dựa trên MPPI/ information-theoretic MPC [8]; project không claim tái tạo mọi theoretical correction term của implementation trong paper.

3.3 Cost bám global path (bản velocity-level)

Minařík et al. xây dựng cost reference-tracking bằng tổng cost input và độ đổi input (Eq. 16), sau đó cộng metric giữa state rollout và reference state theo thời gian (Eq. 17–18) [9]. Metric vị trí của paper là weighted squared error; ngoài vị trí còn có quaternion, vận tốc và body-rate. Đây là nguồn tham khảo cho việc bổ sung tracking cost, không phải bằng chứng rằng mọi thành phần đó đã có trong baseline này.

Global planner/mission cung cấp polyline $\mathcal{P}$ trong ENU. Mỗi chu kỳ, vị trí hiện tại được chiếu lên path để thu progress s_0 ; progress không được lùi so với chu kỳ trước. Với tốc độ reference v_r , từng bước horizon dùng $s_k = s_0 + (k + 1)v_r\Delta t$, từ đó nội suy $\mathbf{p}_k^{ref}$ và lấy sai phân hữu hạn cho $\mathbf{v}_k^{ref}$. Profile paper-only dùng

$$J_{\text{ref}} = \sum_{k=0}^{H-1} \left[w_p \left(\frac{\|\mathbf{p}_k - \mathbf{p}_k^{ref}\|_2}{s_{\text{path}}} \right)^2 + w_v \|\mathbf{v}_k - \mathbf{v}_k^{ref}\|_2^2 \right]. \quad (6)$$

Baseline project vẫn có tùy chọn nearest-polyline

$$J_{\text{polyline}} = w_{\text{path}} \sum_{k=0}^{H-1} \left(\frac{d(\mathbf{p}_k, \mathcal{P})}{s_{\text{path}}} \right)^2, \quad d(\mathbf{p}, \mathcal{P}) = \min_{\text{segment } e \in \mathcal{P}} \|\mathbf{p} - \text{proj}_e(\mathbf{p})\|_2. \quad (7)$$

với d là projection lên segment gần nhất. Path được truyền qua `--global-path`; `--goal` vẫn quyết định active waypoint và điều kiện kết thúc.

Đây là **PROJECT DESIGN DECISION**, không phải reproduction đầy đủ của Eq. 17–18: position/velocity đã time-indexed nhưng chưa có quaternion/body-rate reference, dynamics rigid-body hay

motor feasibility của paper. Polyline cũng không phải collision certificate; path xuyên obstacle tạo xung đột tracking– safety và phải được kiểm tra clearance trước khi chạy.

3.4 Đối chiếu toàn bộ cost với paper

Bảng dưới đây tách các thành phần có cơ sở từ Minařík et al. khỏi heuristic được thêm cho bài toán velocity-level. Paper Eq. 16–18 và Eq. 22 là nguồn tham khảo trực tiếp [9]; profile thử nghiệm mppi_paper_cost_only.yaml chỉ bật các hàng được đánh dấu.

Bảng 3: Đối chiếu cost project và profile paper-only.

Cost hiện tại	Đối chiếu paper	Xử lý trong paper-only
Running goal distance	Paper dùng metric giữa state và reference state theo thời gian; không có active-waypoint heuristic riêng.	Tắt w_{goal} .
Terminal goal distance	Profile đã phạt reference state ở mọi bước; không cần active-waypoint terminal heuristic riêng.	Tắt $w_{terminal}$.
Softplus proximity	Không phải collision indicator Eq. 22.	Tắt; thay bằng $c_{obs}\mathbf{1}_{x \in \mathcal{C}_{obs}}$.
Control effort	Cùng nhóm với Eq. 16, nhưng paper dùng ma trận chéo R .	Dùng $R = \text{diag}(0.01, 0.05, 0.05, 0.10)$ sau ánh xạ velocity-level.
R_{Δ} input-change	Eq. 16 phạt $\Delta u_j^T R_{\Delta} \Delta u_j$.	Dùng $R_{\Delta} = \text{diag}(0.05, 0.10, 0.10, 0.30)$ trên chuỗi action.
$w_{du}(u - v)$	Không phải $\Delta u_j = u_{j+1} - u_j$ của Eq. 16.	Tắt; thay bằng R_{Δ} tính trên successive controls trong terminal callback.
Yaw heuristic	Không có term tương ứng trực tiếp trong Eq. 16–18.	Tắt.
Time-indexed global path	Paper dùng full reference state theo thời gian.	Bật p_k^{ref} và v_k^{ref} ; thiếu attitude/body-rate.

Trong profile paper-only, phần input-change được tính theo

$$J_{\Delta u} = \sum_{j=0}^{H-2} \Delta u_j^T R_{\Delta} \Delta u_j, \quad \Delta u_j = u_{j+1} - u_j, \quad (8)$$

trên toàn bộ chuỗi control khả thi của mỗi rollout tại terminal callback. Vì action của baseline là $[v_x, v_y, v_z, \dot{\psi}]$ thay vì $[F_t, \omega_d]$, ma trận R_{Δ} của paper chỉ được ánh xạ theo thứ tự control này; đây là adaptation có chủ đích, không phải khẳng định tương đương về vật lý.

4 PA-MPPI v0

4.1 Occupancy map ba trạng thái

PA-MPPI cần phân biệt vùng đã biết và chưa biết. Bản đồ voxel dùng quy ước

$$G(\mathbf{p}) \in \{-1, 0, 1\}, \quad -1 : \text{unknown}, 0 : \text{free}, 1 : \text{occupied}. \quad (9)$$

Đây là ba trạng thái và unknown-space safety policy từ paper [3]; backend dưới đây là implementation riêng của project.

Với mỗi LiDAR endpoint, voxel trên ray được đánh dấu free và endpoint được đánh dấu occupied. Occupied được ghi sau để không bị ray lân cận xóa. Vì point cloud Gazebo sau downsample chỉ giữ hit endpoint, mapper hiện đánh dấu một vùng gần sensor là free. Đây là giả thiết thực dụng cho prototype, không thay thế việc tích hợp đầy đủ max-range/miss ray của sensor thật.

4.2 Perception-aware objective

Khi chưa có line-of-sight trực tiếp tới goal, terminal cost được bổ sung bằng

$$\ell_{\text{perception}} = c_{\text{PoI}}(1 - \langle \hat{\mathbf{x}}_B, \hat{\mathbf{d}}_{\text{goal}} \rangle)^2 + c_{\text{occ}} \mathbf{1}_{G(\mathbf{r}(t^*))=1} + c_{\text{unk}} \mathbf{1}_{G(\mathbf{r}(t^*))=-1}. \quad (10)$$

Thành phần đầu hướng trục camera/body-x về goal. Hai thành phần sau phạt ray đụng occupied voxel và thưởng endpoint nhìn vào unknown frontier. Collision cost coi mọi voxel không phải free là không an toàn. Perception cost được tắt khi goal đã nhìn thấy trực tiếp, phù hợp cấu trúc hai pha trong [3].

PA-MPPI v0 vẫn dùng velocity-level dynamics. Mục đích của v0 là kiểm chứng occupancy/perception integration trên interface ổn định trước khi hạ xuống tầng thrust/body rate. Đây là tầng B và là **PROJECT DESIGN DECISION**: unit test xác nhận v0 dùng đúng cùng baseline dynamics, nhằm tách câu hỏi perception-aware planning khỏi câu hỏi low-level thrust/body-rate.

5 Rigid-body PA-MPPI experimental

Đây là tầng C. Control space tiến gần formulation của paper hơn [3], nhưng prediction model, mapper, rollout scale và validation không khớp hoàn toàn; vì vậy tên đúng là “experimental”, không phải full reproduction.

5.1 State, control và hệ trục

Nhánh experimental dùng

$$\mathbf{x} = [\mathbf{p}_W^\top, \mathbf{q}_{WB}^\top, \mathbf{v}_W^\top, \boldsymbol{\omega}_B^\top]^\top \in \mathbb{R}^{13}, \quad \mathbf{u} = [c, \boldsymbol{\omega}_{cmd}^\top]^\top \in \mathbb{R}^4. \quad (11)$$

Quaternion $\mathbf{q}_{WB} = [q_w, q_x, q_y, q_z]$ biến đổi vector body FLU của Gazebo sang world ENU. Control c là collective thrust theo newton; $\boldsymbol{\omega}_{cmd}$ là body rate FLU. Trước khi gửi MAVLink, phép đổi

$$[p, q, r]_{FRD} = [p, -q, -r]_{FLU} \quad (12)$$

đưa body rate về quy ước FRD của ArduPilot.

5.2 Dynamics rời rạc

Với $\mathbf{e}_3 = [0, 0, 1]^\top$, gia tốc trong ENU là

$$\dot{\mathbf{v}}_W = \frac{c}{m} R(\mathbf{q}_{WB}) \mathbf{e}_3 + [0, 0, -g]^\top. \quad (13)$$

Quaternion kinematics và body-rate tracking được mô hình hóa bởi

$$\dot{\mathbf{q}}_{WB} = \frac{1}{2} \mathbf{q}_{WB} \otimes [0, \boldsymbol{\omega}_B], \quad (14)$$

$$\boldsymbol{\omega}_{k+1} = \boldsymbol{\omega}_k + \alpha_\omega (\boldsymbol{\omega}_{cmd,k} - \boldsymbol{\omega}_k), \quad \alpha_\omega = \min(\Delta t / \tau_\omega, 1). \quad (15)$$

Forward Euler được dùng cho velocity, quaternion và rate; position dùng thêm $\frac{1}{2}a\Delta t^2$. Quaternion được chuẩn hóa sau mỗi bước. Model hiện mô tả response của closed-loop body-rate controller bằng first-order lag. Phần vật lý chỉ gồm thrust acceleration và quaternion kinematics; phần inner-loop là approximation của ArduPilot. Nó chưa mô phỏng torque, moment of inertia, rotor allocation, drag, battery hoặc motor lag tách biệt như dynamics trong paper [3]. Đây là reduced closed-loop prediction model; không có inertia value nào được suy đoán.

Khối lượng 2.10 kg được cộng từ SDF: iris/IMU/bốn rotor $1.5 + 0.15 + 4(0.025) = 1.75$ kg, gimbal 0.23 kg và sensor pod 0.12 kg. Giá trị phụ thuộc đúng include tree; center of mass và inertia tổng chưa được nhận dạng. Hover force là $c_h = mg$. Ánh xạ từ newton sang thrust chuẩn hóa dùng

$$T_{norm} = T_{hover,norm} \frac{c}{mg}, \quad (16)$$

với $T_{hover,norm} = 0.38$. Đây là calibration assumption từ historical log logs/00000002.BIN, trong đó ArduPilot từng học MOT_THST_HOVER khoảng 0.363–0.386. Nó phải được revalidate nếu đổi mass/model/propeller/plugin.

5.3 Frame contract

MAVLink LOCAL_NED quy định north/east/down và z altitude âm [4]. Các transform không được xem là đúng chỉ vì unit test pass; convention và runtime data source cũng phải khớp.

Bảng 4: Interface contract cho coordinate frames.

Source → destination	Convention/units	Code	Evidence/status
Gazebo world ENU → LOCAL NED	$(E, N, U) \rightarrow (N, E, -U)$, m và m/s	enu_to_ned_vel, ned_to_enu_pos	Spec + round-trip/sign test; live origin pending.
Body FLU → body FRD	$(x, y, z) \rightarrow (x, -y, -z)$	point transform; body_flu_rates_to_frd	deterministic test.
Yaw NED → ENU	$\psi_E = \pi/2 - \psi_N$	yaw_ned_to_enu	cardinal test.
Yaw-rate ENU → NED	$\dot{\psi}_N = -\dot{\psi}_E$	yaw_enu_to_ned_rate	sign test.
Gazebo quaternion → ENU rotation	message xyzw; internal wxyz, Hamilton	quat_to_rot	identity/norm tests.
LiDAR/camera → body/world	SDF offset; odom full attitude	scan_to_world_enu	source review; live frame pending.

Odometry.twist.angular hiện được giả định là body FLU. Exact installed Gazebo plugin convention và một maneuver có dấu vẫn phải xác nhận trước rigid live gate; đây là **UNVERIFIED/PENDING**.

6 Tích hợp MAVLink và các điều kiện an toàn

6.1 Velocity target

Vanilla và PA-MPPI v0 gửi SET_POSITION_TARGET_LOCAL_NED. Type mask 1479 bỏ position, acceleration và yaw; velocity cùng yaw-rate vẫn được sử dụng. Mask đã được đối chiếu với enum của dialect ardupilotmega và handler ArduCopter target commit [4, 2]. Dạng symbolic là $1 + 2 + 4 + 64 + 128 + 256 + 1024 = 1479$; velocity và yaw-rate ignore bits đều clear. Fallback velocity-only thêm yaw-rate-ignore 2048, tạo mask 3527.

6.2 Goal tương tác từ RViz

Node có thể subscribe geometry_msgs/PoseStamped từ tool 2D Goal Pose trên topic /goal_pose. Goal chỉ được chấp nhận khi frame là odom; frame khác bị từ chối vì implementation chưa tự áp dụng

TF. Mỗi click thay route đang chạy, xóa terminal/recovery latch, reset output conditioner và MPPI warm start. Vì tool 2D thường phát $z = 0$, project bỏ giá trị z trong message và giữ altitude hiện tại, hoặc dùng altitude ENU cố định từ `--rviz-goal-altitude`. Đây là **PROJECT INTERFACE DECISION** nhằm tránh một thao tác XY vô tình tạo lệnh hạ xuống đất.

Trong paper profile, mỗi click tạo reference thẳng từ state hiện tại tới goal. Do đó giao diện này thay việc nhập Fly To/tọa độ thủ công, nhưng không phải global planner: operator vẫn phải đặt waypoint trung gian trong dead-end, U-wall hoặc path bị che khuất. Callback topic và logic thay route đã được kiểm tra offline; chuyển bay Gazebo điều khiển hoàn toàn bằng click còn PENDING.

6.3 Thrust và body-rate target

Rigid branch gửi SET_ATTITUDE_TARGET với mask 128: attitude quaternion bị bỏ qua, cả ba body rate và thrust được giữ lại. ArduCopter không chấp nhận body-rate vector thiếu một phần. Mask được dựng từ ATTITUDE_TARGET_TYPMASK_ATTITUDE_IGNORE, không phải magic decimal [4].

Có discrepancy phải giữ công khai: trang “Copter Commands in Guided Mode” hiện nói ba body-rate fields không được hỗ trợ và mask nên bằng 7 [5]. Ngược lại, source target commit f808f78..., GCS_MAVLink_Copter.cpp:892--966, nhận quaternion, nhận đủ cả ba rates, hoặc cho phép cả ba cùng ignored; partial rate vector làm handler gọi hold [2]. Vì project khóa commit này, implementation theo source behavior nhưng không che giấu khác biệt tài liệu.

Trường thrust chỉ mang nghĩa normalized thrust khi bit 3 của tham số GUID_OPTIONS được bật. Vì vậy cấu hình phải đặt giá trị này bằng 8. Nếu giữ giá trị 0, ArduPilot diễn giải cùng trường thành climb-rate. Node đọc lại parameter qua MAVLink và từ chối phát lệnh nếu giá trị không bằng 8. Bằng chứng source là mode.h:1208--1217, mode_guided.cpp:651--655 và handler lines 943--965 tại target commit [2].

Các gate bổ sung gồm:

- bắt buộc cờ `--experimental-attitude-control`;
- chỉ hỗ trợ Gazebo odometry ở giai đoạn hiện tại;
- control frequency tối thiểu 20 Hz, khuyến nghị 50 Hz;
- thrust được giới hạn trong $[0, 1]$ trước khi gửi;
- NaN/Inf trong rate hoặc thrust làm packet bị từ chối;
- planner vượt timeout cấu hình thì không phát control mới;
- stale, hard-brake, reached và Ctrl+C chuyển sang velocity-zero target, không phát zero thrust.

File config/mppi_attitude_sitl.parm đặt GUID_OPTIONS=8, MOT_THST_HOVER=0.38 và tắt hover learning cho thí nghiệm tái lập.

7 Thiết kế kiểm thử

7.1 Automated tests

Test suite dùng unittest và chạy trong env ardupilot-rviz. Các nhóm test chính được liệt kê trong Bảng 5.

7.2 Offline obstacle scenario

Obstacle giả lập là một mặt stack container tại $x = 12$ m, $y \in [-6, 6]$ m và $z \in [0, 24]$ m. UAV bắt đầu tại $(0, 0, 20)$ m và bám tuyến waypoint $(16, 10, 20) \rightarrow (30, 0, 20)$. Điều kiện đạt gồm:

$$\|p_T - p_{goal}\| < 1 \text{ m}, \quad d_{min} \geq margin + 1 \text{ m} = 5 \text{ m}. \quad (17)$$

Bảng 5: Phạm vi automated tests.

Nhóm	Nội dung
Frame và MAVLink	ENU/NED, FLU/FRD, quaternion cơ sở, velocity mask 1479, attitude mask 128, partial-rate rejection, GUID gate, NaN/Inf và saturation.
Planner safety	stale state, hard brake, timeout, reached, waypoint và diagnostics.
Occupancy/PA-MPPI	unknown/free/occupied, ray traversal, LOS, perception reward, unknown collision và identical baseline dynamics.
Rigid-body	hover equilibrium, quaternion norm, body-rate response, thrust mapping và một MPPI optimizer step.
Closed-loop clear-air	80 chu kỳ tới goal 1 m, kiểm tra sai số đích và dải cao độ.
Configuration	YAML runtime fields và ưu tiên CLI override.

Waypoint trung gian đóng vai trò global direction tối thiểu. Nó phá tính đối xứng trái/phải trước obstacle rộng, một trường hợp mà local MPPI có thể tạo trung bình control gần bằng không.

8 Kết quả

Bảng 6: Kết quả định lượng cập nhật ngày 13/09/2026.

Thí nghiệm	Kết quả	Chỉ số
Automated test suite	Đạt offline	40/40 test; exact command lưu trong changelog.
Vanilla MPPI obstacle	Đạt offline	seed 7; final error 0.93 m; minimum clearance 6.30 m.
PA-MPPI v0 obstacle	Đạt offline	seed 7; final error 0.92 m; minimum clearance 6.11 m.
Global-path reference smoke	Đạt offline	seed 7; final error 0.23 m; minimum point clearance 5.85 m.
Paper-reference benchmark	Đạt offline	3 map $\times$ seed 7/11/19: 9/9 terminal; point-radius analytical adapter.
Right-angle Gazebo/SITL	Đạt, một seed	128 cycles; final error 0.235 m; path 23.15 m; min point clearance 2.19 m.
Narrow-gate Gazebo/SITL	Đạt, một seed	192 cycles; final error 0.230 m; path 35.50 m; min point clearance 2.07 m.
Slalom Gazebo/SITL	Đạt, một seed	193 cycles; final error 0.220 m; path 37.67 m; min point clearance 1.30 m.
Rigid hover dynamics	Đạt	position/velocity numerical error bằng 0; quaternion norm bằng 1.
Rigid closed-loop unit gate	Đạt offline	80 cycles; asserted final error < 0.5 m và altitude trong 1.5–2.5 m; seed 7.
Rigid live Gazebo hover	Pending	Chưa chạy nhánh rigid body-rate/thrust trong Gazebo.

Ba run velocity-level live đều giữ cao độ trong khoảng 19.81–20.41 m và không ghi deadline miss ở target 10 Hz. Đây chỉ là một seed trên mỗi map. Clearance là khoảng cách tâm odometry–điểm LiDAR, chưa trừ footprint; slalom đạt 1.30 m, thấp hơn collision radius rollout 1.5 m, nên chưa thể suy ra safety. Rigid branch giữ hover trong internal model nhưng chưa có benchmark live/edge.

JSONL logger hiện ghi timestamp/state age, position, velocity, quaternion, body rates, nominal/sent control, thrust, map status, clearance, goal/LOS, costs, rollout statistics, selected trajectory, deadline/saturation/failsafe và seed. `scripts/plot_mppi_experiment.py` chuẩn bị các plot trajectory, altitude, velocity, commanded/measured rates, thrust, clearance, goal distance, latency và MPPI-vs-PA

comparison; không synthetic-fill field bị thiếu.

9 Thảo luận

9.1 Mức độ đáp ứng yêu cầu mentor

Kiến trúc hiện tại đáp ứng ý chính: obstacle avoidance và local planning nằm ở companion computer; ArduPilot không cần obstacle ID và chỉ bám setpoint. Vanilla MPPI là baseline chạy được. PA-MPPI v0 chứng minh occupancy/perception cost. Rigid branch đã đưa control space tới thrust/body rate như hướng của bài báo, nhưng mới ở mức offline-verified experimental path.

9.2 Khoảng cách tới bản triển khai trong bài báo

Bảng 7: So sánh phạm vi hiện tại với PA-MPPI [3].

Feature	Paper	Current implementation	Status/reason
State/control	p, q, v, ω ; thrust/rates	velocity p, v, ψ or rigid p, q, v, ω	v0 reduced; rigid experimental.
Prediction	Full quad dynamics incl. torque/inertia	point-mass lag or thrust + first-order rate lag	Not full reproduction.
Map	ROG-Map; 3 states; 0.1 m, $5 \times 5 \times 2$ m	fixed NumPy/Torch grid; near-field heuristic	Backend differs.
Collision unknown	$G(p) \neq 0$ penalized	same policy when map exists	Implemented offline.
Perception/axis	camera/body-x + goal ray	heading-x v0; body-x rigid	v0 approximation.
Ray/LOS phase	terminal DDA; two phases	terminal sampled ray; LOS switch	Different ray implementation.
Rollouts/horizon	17,500 / 15	500/30 default; 128/20 ablation; 512/20 rigid	Reduced compute scale.
Timing	$\Delta t_{pred} = 0.1s$, 50 Hz control	velocity 0.1 s/10 Hz; rigid 0.05 s/50 Hz target	Live deadline pending.
Framework	JAX + Agilicious	PyTorch + pytorch-mppi	Differs.
Mapping input	Depth + ROG-Map	Gazebo LiDAR hit endpoints	Sensor model differs.
Environment	Flightmare; C-wall/hole/4-wall; HIL/real	simple offline wall; Gazebo prepared	Matching evaluation pending.
Real flight	Demonstrated in paper	None	Pending hardware.

Do các khác biệt này, nhánh hiện tại phải được gọi là “PA-MPPI v0” hoặc “rigid-body PA-MPPI experimental”, không phải reproduction đầy đủ.

9.3 Rủi ro kỹ thuật còn lại

- Hover thrust phụ thuộc model, propeller và battery model. Giá trị 0.38 phải hiệu chỉnh lại nếu SDF hoặc motor configuration thay đổi.
- Khối lượng 2.10 kg là tổng xấp xỉ từ model components; center of mass và inertia tổng chưa được nhận dạng chính xác.
- Gazebo odometry angular velocity cần được xác nhận thực nghiệm đúng body FLU trong mọi cấu hình plugin.

- Nhánh MAVLink state hiện chỉ đủ cho velocity planner; rigid branch chưa chuyển quaternion/rate từ estimator ArduPilot sang frame nội bộ.
- Sensor dropout, latency, obstacle động và estimator noise chưa được đưa vào Monte Carlo evaluation.
- Velocity-zero fallback đổi GUIDED submode nhưng không thay thế một independent safety controller hoặc người giám sát khi thử attitude control.

10 Lộ trình kiểm chứng tiếp theo

1. **Hover gate:** takeoff bằng velocity/position controller, bật rigid node với goal ngắn trong không gian trống, theo dõi altitude, attitude, thrust target và mode.
2. **Body-rate gate:** áp goal lệch 1 m theo từng trục, giới hạn rate, đo tracking delay để cập nhật τ_ω .
3. **Clear-air route:** bay nhiều waypoint không obstacle; đo final error, overshoot, thrust saturation và deadline miss.
4. **Static obstacle:** bật occupancy/perception cost với một stack container, sau đó tăng độ phức tạp môi trường.
5. **Ablation:** so sánh vanilla, PA-MPPI v0 và rigid PA-MPPI bằng cùng seed/scenario theo success rate, clearance, path length, smoothness và compute p95.
6. **Edge computer:** tích hợp timestamp và full attitude transform; sau đó benchmark tài nguyên và nhiệt độ.
7. **Hardware gate:** HIL/tether test trước; chỉ flight test khi có geofence, mode switch và pilot override đã kiểm chứng.

Simple wall có waypoint trung gian chỉ là baseline regression, không chứng minh lợi ích perception. Fair experiment tương lai dùng C-wall/U-shaped occlusion với cùng start, goal, world, dynamics, limits, horizon, base cost và seed; chỉ bật/tắt perception terms. Hai config cố định là config/ablation_mppi_unknown.yaml và config/ablation_pa_mppi_unknown.yaml. Metrics bắt buộc gồm success rate, time-to-goal, path length, minimum clearance, control smoothness, planner cycles, compute mean/p95/worst, deadline misses, mapped free space và time tới lần đầu goal visible. Toàn bộ gate/abort/log name nằm trong docs/GAZEBO_RUN_CHECKLIST.md; hiện chưa gate nào được đánh dấu pass.

11 Kết luận

Repository đã chuyển được obstacle avoidance khỏi ArduPilot và hình thành một pipeline local planning có thể thay đổi model/control level mà không sửa tầng perception hoặc mission interface. Vanilla MPPI và PA-MPPI v0 đã chạy qua obstacle scenario offline. Rigid-body experimental đã bổ sung quaternion, thrust/body-rate control, SET_ATTITUDE_TARGET, calibration và safety gate, đồng thời vượt qua 23 automated tests/offline gates. Kết quả đủ để chuyển sang hover gate trong Gazebo, nhưng chưa đủ để tuyên bố hoàn thành rigid PA-MPPI hoặc triển khai trên quadrotor thật.

A Lệnh tái lập

A.1 Automated tests

```
KMP_DUPLICATE_LIB_OK=TRUE OMP_NUM_THREADS=1 \
/opt/miniconda3/envs/ardupilot-rviz/bin/python \
-m unittest discover -s tests -p 'test_mppi*.py' -v
```

A.2 Rigid-body offline gate

```
KMP_DUPLICATE_LIB_OK=TRUE OMP_NUM_THREADS=1 \
```

```
/opt/miniconda3/envs/ardupilot-rviz/bin/python \
scripts/mppi_velocity_avoidance.py \
--planner rigid-pa-mppi \
--config mppi_ardupilot/rigid_pa_mppi_config.yaml \
--sim-test
```

A.3 Rigid-body live gate trong Gazebo

```
MAVLINK20=1 KMP_DUPLICATE_LIB_OK=TRUE OMP_NUM_THREADS=1 \
/opt/miniconda3/envs/ardupilot-rviz/bin/python \
scripts/mppi_velocity_avoidance.py \
--planner rigid-pa-mppi \
--config mppi_ardupilot/rigid_pa_mppi_config.yaml \
--experimental-attitude-control \
--state-source odom --goal "1,0,20" \
--mav tcp:127.0.0.1:5762 \
--diag-jsonl output/log/rigid_hover_gate.jsonl
```

Tài liệu tham khảo

- [1] MAVLink Project. *MAVLink ArduPilotMega Dialect: OBSTACLE_DISTANCE_3D*, 2026. <https://mavlink.io/en/messages/ardupilotmega.html>; accessed 2026-09-10. Vendored mavlink commit 71d925850d7ddb01d0e56defc1046f9d9d3bf8ae.
- [2] ArduPilot Project. Ardupilot source tree at commit f808f78ce5a518ca96f2fb36420608b2b6254367. Git source repository, 2026. <https://github.com/ArduPilot/ardupilot/tree/f808f78ce5a518ca96f2fb36420608b2b6254367>; files cited in text; accessed 2026-09-10.
- [3] Yifan Zhai, Rudolf Reiter, and Davide Scaramuzza. PA-MPPI: Perception-aware model predictive path integral control for quadrotor navigation in unknown environments. *IEEE Robotics and Automation Letters*, 11(3):3804–3811, 2026. DOI: 10.1109/LRA.2026.3662653; available at https://rpg.ifi.uzh.ch/docs/RAL26_Zhai.pdf.
- [4] MAVLink Project. *MAVLink Common Message Set: SET_POSITION_TARGET_LOCAL_NED and SET_ATTITUDE_TARGET*, 2026. <https://mavlink.io/en/messages/common.html>; accessed 2026-09-10. Vendored mavlink commit 71d925850d7ddb01d0e56defc1046f9d9d3bf8ae.
- [5] ArduPilot Project. *Copter Commands in Guided Mode*, 2026. <https://ardupilot.org/dev/docs/copter-commands-in-guided-mode.html>; accessed 2026-09-10.
- [6] ArduPilot Project. *Simple Object Avoidance*, 2026. <https://ardupilot.org/copter/docs/common-simple-object-avoidance.html>; accessed 2026-09-10.
- [7] ArduPilot Project. *Object Avoidance with Bendy Ruler*, 2026. <https://ardupilot.org/copter/docs/common-aa-bendyruler.html>; accessed 2026-09-10.
- [8] Grady Williams, Paul Drews, Brian Goldfain, James M. Rehg, and Evangelos A. Theodorou. Information-theoretic model predictive control: Theory and applications to autonomous driving. *IEEE Transactions on Robotics*, 34(6):1603–1622, 2018. DOI: 10.1109/TRO.2018.2865891; arXiv:1707.02342.
- [9] Michal Minařík, Robert Pěnička, Vojtěch Vonásek, and Martin Saska. Model predictive path integral control for agile unmanned aerial vehicles. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024. arXiv:2407.09812v1; accessed 2026-09-11.